Pioneer River Water LHS-1

Direct Step 0-7 Calculation and Detailed Code Explanation

Purpose. This report follows one compact Python script from the model files to the final Main-CV water-storage change. The emphasis is not on adding separate verification routines. Instead, each code block is explained in detail so that the origin, indexing, unit and physical meaning of every quantity used in the calculation are explicit.

$$H_i = d_i + \eta_i$$

$$H_e = \frac{H_1 + H_2 + H_3}{3}$$

$$V_{CV} = \Sigma V_e, \quad \Delta V = V(T_1) - V(T_0)$$

The direct calculation uses only hgrid.gr3, the Main-CV element list and out2d elevation/wet-dry output. zCoordinates is not used in the direct calculation; it was used only as an independent check of the bathymetry/elevation sign convention.

1. Files, run and saved states

```
RUN = Path(
    r"R:\Estuary_SCHISM-Q8213\9.0 Developed models\Pioneer"
    r"\Pioneer_17_waterlevel_flow_calibration_AprJun2025_BudgetE"
)
HGRID = RUN / "hgrid.gr3"
CV_FILE = RUN / "cv_pioneer_nested_M4_N1_elements.txt"
OUT = RUN / "outputs"

# Exact saved states used for the 30-day storage calculation.
STATES = {
    "T0": (OUT / "out2d_1.nc", 0, 900.0),
    "T1": (OUT / "out2d_5.nc", 191, 2592000.0),
}

R_EARTH = 6371008.8  # m
```

RUN points to the completed Pioneer BudgetE simulation. HGRID is the static SCHISM horizontal grid. CV_FILE contains the element IDs that define the Main control volume. OUT is the directory containing the saved SCHISM outputs.

The calculation uses two already-established saved states. T_0 is out2d_1.nc record 0 at 900 s. T_1 is out2d_5.nc record 191 at 2,592,000 s. These are the beginning and end snapshots used for the 30-day storage change.

Item	Source	What is used
Main CV	cv_pioneer_nested_M4_N1_elements.txt	16,186 element IDs
Static mesh	hgrid.gr3	node longitude, latitude, bathymetric depth and element-node connectivity
T_0 state	out2d_1.nc, record 0	elevation and dryFlagElement at 900 s
T_1 state	out2d_5.nc, record 191	elevation and dryFlagElement at 2,592,000 s

2. Step 0 - Read hgrid.gr3

```
def read_grid():
    with HGRID.open("r", encoding="utf-8", errors="ignore") as f:
        title = f.readline().strip()
        ne, nn = map(int, f.readline().split()[:2])

        lon = np.full(nn + 1, np.nan)
        lat = np.full(nn + 1, np.nan)
        depth = np.full(nn + 1, np.nan)     # node ID -> bathymetric depth d [m]

        for _ in range(nn):
            p = f.readline().split()
            nid = int(p[0])
            lon[nid] = float(p[1])
            lat[nid] = float(p[2])
            depth[nid] = float(p[3])

        elems = [None] * (ne + 1)          # element ID -> its node IDs
        for _ in range(ne):
            p = f.readline().split()
            eid, nv = int(p[0]), int(p[1])
            elems[eid] = tuple(map(int, p[2:2 + nv]))

    return title, lon, lat, depth, elems
```

The first two lines of hgrid.gr3 give the grid title and the numbers of elements and nodes. The next NN lines are node records. For each node, the script reads: node ID, x coordinate, y coordinate and the fourth column d. In this Pioneer grid x and y are longitude and latitude in degrees, while d is bathymetric depth in metres.

The arrays lon, lat and depth are deliberately created with length NN+1. Index 0 is unused. This makes the Python index identical to the SCHISM node ID: depth[39107] means 'the depth stored for node 39107'.

After the node section, hgrid.gr3 contains the element connectivity. Each element record gives an element ID, the number of vertices, and the node IDs belonging to that element. The script stores this as elems[eid]. Therefore, for a triangular element, elems[eid] returns exactly three model node IDs.

Object in code	**Spatial basis**	**Unit**	**Meaning**
lon[nid]	node	degree E	node longitude
lat[nid]	node	degree N	node latitude
depth[nid]	node	m	SCHISM bathymetric depth d; positive-down convention
elems[eid]	element to nodes	-	tuple of node IDs defining that element

3. Step 0 - Read the Main-CV element IDs

```
def read_main_cv():
    eids = []
    for line in CV_FILE.read_text(encoding="utf-8", errors="ignore").splitlines():
        s = line.strip()
        if s and not s.startswith("#"):
            eids.append(int(float(s.split()[0])))
    return np.asarray(sorted(set(eids)), dtype=int)
```

The Main CV is represented as a list of SCHISM element IDs. The script reads the first value on each non-empty, non-comment line, removes duplicates and sorts the IDs. At this point the calculation has a set of element IDs, not node IDs. These element IDs are the objects that will be integrated.

The important mapping used later is therefore: Main-CV element ID -> hgrid connectivity -> the three node IDs belonging to that element.

Actual walkthrough output A - mesh and Main-CV objects	
Variable / check	Actual Pioneer value
Grid title	hgrid_deepen.gr3
Mesh size	130,884 elements; 67,762 nodes
lon / lat / depth array shape	(67,763,) each; index 0 intentionally unused
First node	lon[1] = 149.091928; lat[1] = -21.147282; depth[1] = 1.000000 m
Main-CV array	cv.shape = (16,186,); min eid = 149; max eid = 130,878
First five Main-CV eids	[149, 150, 152, 153, 156]
Representative connectivity	elems[149] = (7917, 8016, 8107)

4. Step 3 - Calculate the horizontal area of each Main-CV element

```
def metric_xy(lon, lat):
    lon0 = np.nanmean(lon[1:])
    lat0 = np.nanmean(lat[1:])
    x = R_EARTH * math.cos(math.radians(lat0)) * np.deg2rad(lon - lon0)
    y = R_EARTH * np.deg2rad(lat - lat0)
    return x, y


def triangle_area(eid, elems, x, y):
    n1, n2, n3 = elems[eid]
    return 0.5 * abs(
        (x[n2] - x[n1]) * (y[n3] - y[n1])
        - (x[n3] - x[n1]) * (y[n2] - y[n1])
    )
```

The hgrid coordinates cannot be inserted directly into a planar area formula because they are longitude and latitude in degrees. metric_xy uses a local equirectangular approximation to convert every node to Cartesian x/y coordinates in metres. The longitudinal distance is multiplied by cos(lat0) to account for the convergence of meridians away from the equator. The conversion is centred on the mean longitude and latitude of the mesh, and R_{EARTH} = 6,371,008.8 m supplies the length scale.

triangle_area then starts from one element ID. elems[eid] returns n_1, n_2 and n_3. The metric x/y coordinates of those three nodes are inserted into the standard triangle cross-product formula. Because x and y are metres, the result A_e is square metres.

$$\mathrm{A_e} = \frac{1}{2}\mathrm{abs}((\mathrm{x_2} - \mathrm{x_1})(\mathrm{y_3} - \mathrm{y_1}) - (\mathrm{x_3} - \mathrm{x_1})(\mathrm{y_2} - \mathrm{y_1}))$$

Area is static: it depends only on the horizontal mesh, not on T_0 or T_1. Therefore the script calculates one area for each of the 16,186 Main-CV elements once and reuses the same area at both saved states.

Actual walkthrough output B - local coordinates and triangle area	
Variable / check	Actual Pioneer value
Mesh mean longitude / latitude	lon0 = 149.204524109 deg; lat0 = -21.214878251 deg
cos(lat0)	0.932229865114
Element 149 nodes	(7917, 8016, 8107)
Node 7917 local x / y	-4,724.079970 m / +8,722.726001 m
Node 8016 local x / y	-4,708.116427 m / +8,703.378058 m
Node 8107 local x / y	-4,698.268787 m / +8,726.284244 m
Area of element 149	278.097754 m²
Main-CV total plan area	4,575,495.046715 m²

5. Read one out2d saved state

```
def read_state(path, record, expected_time, ne, nn):
    with Dataset(path) as ds:
        time = float(ds.variables["time"][record])
        eta0 = np.ma.filled(
            ds.variables["elevation"][record, :], np.nan
        ).astype(float)
        dry0 = np.ma.filled(
            ds.variables["dryFlagElement"][record, :], np.nan
        ).astype(float)

    if abs(time - expected_time) > 2.0:
        raise RuntimeError(f"Unexpected saved time in {path.name}: {time}")

    # Convert NetCDF's 0-based arrays to 1-based arrays so model node/element
    # IDs can be used directly: eta[node_id], dry[element_id].
    eta = np.full(nn + 1, np.nan)
    dry = np.full(ne + 1, np.nan)
    eta[1:] = eta0
    dry[1:] = dry0

    return time, eta, dry
```

For one saved state the script reads three things from out2d: time, elevation and dryFlagElement. elevation is node-based. dryFlagElement is element-based. This distinction is critical because the same element ID cannot be used directly to index a node variable.

The raw NetCDF arrays use Python/NetCDF zero-based positions, while SCHISM node and element IDs in hgrid start at 1. To make later indexing explicit, the script creates new arrays with one unused position at index 0. After eta[1:] = eta0, eta[node_id] is the elevation for that exact SCHISM node. After dry[1:] = dry0, dry[element_id] is the dry flag for that exact SCHISM element.

Variable	**Basis**	**Unit / values**	**Physical meaning**
elevation	node	m; positive upward	water-surface elevation η
dryFlagElement	element	0 = wet; 1 = dry	whether the whole element is active as water at this saved state
time	saved record	s elapsed	model time of the selected snapshot

Actual walkthrough output C - saved-state arrays	
Variable / check	Actual Pioneer value
T0 source	out2d_1.nc; record 0; time = 900 s
T0 elevation example	eta[1] = +0.002882711 m
T0 Main-CV dry state	12,111 wet; 4,075 dry; total = 16,186
T1 source	out2d_5.nc; record 191; time = 2,592,000 s
T1 elevation example	eta[1] = +0.957436442 m
T1 Main-CV dry state	15,562 wet; 624 dry; total = 16,186
Array indexing	eta.shape = (67,763,); dry.shape = (130,885,); index 0 = NaN/unused

6. Steps 1, 2, 4, 5 and 6 - The element-by-element calculation

```
def main_cv_volume(cv, elems, depth, area, eta, dry):
    total = 0.0
    wet_count = 0
    dry_count = 0

    # Start from an element ID in the Main CV.
    for eid in cv:

        # STEP 4: dryFlagElement is element-based.
        # 0 = wet; 1 = dry.
        if dry[eid] not in (0.0, 1.0):
            raise RuntimeError(f"Bad dryFlagElement at element {eid}: {dry[eid]}")
        D_e = int(dry[eid])

        if D_e == 1:
            dry_count += 1
            continue

        wet  count += 1

        # Static hgrid connectivity gives:
        # element ID -> the three node IDs belonging to that triangle.
        n1, n2, n3 = elems[eid]

        # STEP 1: node water depth H_i = d_i + eta_i [m].
        H1 = depth[n1] + eta[n1]
        H2 = depth[n2] + eta[n2]
        H3 = depth[n3] + eta[n3]

        # STEP 2: triangle mean water depth [m].
        H_e = (H1 + H2 + H3) / 3.0

        # STEP 5: wet-element water volume [m3].
        V  e = area[eid] * H  e

        # STEP 6: discrete spatial integration over the Main CV.
        total += V_e

    return total, wet_count, dry_count
```

This function is the core of the calculation. The loop begins with one Main-CV element ID eid. The first time-dependent question is whether this element is wet or dry, so dry[eid] is read before any nodal water-depth calculation.

Step 4 - wet/dry decision. dryFlagElement is already element-based, so no node mapping is needed to answer this question. If dry[eid] = 1, the element contains no active water volume in this calculation. The script counts it as dry and immediately continues to the next element. No nodal elevation or water depth is calculated for that dry element.

If dry[eid] = 0, the element is wet. Only then does the script use elems[eid] to retrieve n_1, n_2 and n_3, the three node IDs that define this triangle. This is the exact element-to-node connection in the model grid.

Step 1 - node water depth. depth[n_1] and eta[n_1] refer to the bathymetric depth and free-surface elevation at the same model node n_1. The same operation is repeated for n_2 and n_3.

$$\mathrm{H_i} = \mathrm{d_i} + \eta_\mathrm{i}$$

Here d_i is the bathymetric depth read from hgrid.gr3 and η_i is the free-surface elevation read from out2d. For this Pioneer run, the sign convention has been independently confirmed from zCoordinates: $z_{bottom} = -d_i$ and $z_{surface} = \eta_i$. Therefore $H_i = z_{surface} - z_{bottom} = \eta_i - (-d_i) = d_i + \eta_i$. H_i is the physical nodal water-column height in metres.

Step 2 - node to element. The three nodal water-column heights are averaged to obtain one mean water depth H_e for the triangular element.

$$\mathrm{H_e} = \frac{\mathrm{H_1} + \mathrm{H_2} + \mathrm{H_3}}{3}$$

Step 5 - element water volume. For a wet element, A_e is in m^2 and H_e is in m, so their product is the element water volume in m^3. Dry elements have already been skipped, which is computationally equivalent to multiplying by (1 - D_e).

$$\mathrm{V_e} = (1 - \mathrm{D_e})\mathrm{A_e}\mathrm{H_e}$$

Step 6 - Main-CV integration. total starts at zero. Each wet element volume V_e is added once. When the loop has visited every Main-CV element, total is the discrete horizontal integral of water volume over the complete Main CV.

$$\mathrm{V_{CV}} = \Sigma \mathrm{V_e}$$

Code operation	Starts from	Returns / uses
D_e = dry[eid]	element ID	element wet/dry state
n1,n2,n3 = elems[eid]	wet element ID	three node IDs
depth[n] + eta[n]	node ID	node water depth H_i [m]
(H1+H2+H3)/3	three node depths	element mean depth H_e [m]
area[eid] * H_e	element area + mean depth	element volume V_e [m³]
total += V_e	each wet element volume	Main-CV volume V_{CV} [m³]

Actual walkthrough output D - representative wet element 149

Quantity	**T_0**	**T_1**
dry[149]	0 (wet)	0 (wet)
d at nodes 7917 / 8016 / 8107 [m]	+0.637169930 / +0.740344440 / +0.295883010	+0.637169930 / +0.740344440 / +0.295883010
η at nodes 7917 / 8016 / 8107 [m]	-0.000025774 / -0.000023432 / -0.000024063	+1.570728660 / +1.569965243 / +1.569500685
H_1 / H_2 / H_3 = d + η [m]	+0.637144156 / +0.740321008 / +0.295858947	+2.207898590 / +2.310309683 / +1.865383695
H_e [m]	+0.557774704	+2.127863989
A_{149} [m²]	278.097754	278.097754
$V_{149} = A_{149} H_e$ [m³]	+155.115893	+591.754196

7. Step 7 - Calculate T_0, T_1 and the storage change

```
def main():
    title, lon, lat, depth, elems = read_grid()
    cv = read_main_cv()
    ne, nn = len(elems) - 1, len(depth) - 1

    # Step 3 is time-independent: calculate Main-CV element areas once.
    x, y = metric_xy(lon, lat)
    area = np.zeros(ne + 1)
    for eid in cv:
        if len(elems[eid]) != 3:
            raise RuntimeError(f"Element {eid} is not a triangle.")
        area[eid] = triangle_area(eid, elems, x, y)

    results = {}

    for name, (path, record, expected_time) in STATES.items():
        time, eta, dry = read_state(path, record, expected_time, ne, nn)
        V, nwet, ndry = main_cv_volume(cv, elems, depth, area, eta, dry)
        results[name] = V

        print(f"{name}: time={time:.0f} s")
        print(f"  wet elements = {nwet:,}")
        print(f"  dry elements = {ndry:,}")
        print(f"  Main-CV volume = {V:,.3f} m3")

    dV = results["T1"] - results["T0"]

    print()
    print(f"hgrid title       = {title}")
    print(f"Main-CV elements  = {len(cv):,}")
    print(f"Main-CV area      = {np.sum(area[cv]):,.3f} m2")
    print(f"V(T0)             = {results['T0']:,.3f} m3")
    print(f"V(T1)             = {results['T1']:,.3f} m3")
    print(f"DeltaV = V1 - V0  = {dV:+,.3f} m3")


if __name__ == "__main__":
    main()
```

main first reads the static grid and the Main-CV element list. It converts the mesh to metric coordinates and calculates all Main-CV triangle areas once. It then repeats the same volume function for T_0 and T_1.

The final Water LHS storage change is the final Main-CV inventory minus the initial Main-CV inventory. A positive ΔV means that the Main CV contains more water at T_1 than at T_0.

$$\Delta V = V(T_1) - V(T_0)$$

8. Numerical result for the 30-day Pioneer Main CV

The numerical values below use the same local equirectangular geometry as Step 3 with R_{EARTH} = 6,371,008.8 m, matching the current production water-budget script 110. The first walkthrough execution used 6,371,000.0 m and returned ΔV = 7,201,230.547 m³. Under this projection x and y scale linearly with R, so area and every A_eH_e volume scale exactly with R^2; bathymetry, elevation, dry flags and water-column heights are unchanged. Harmonising the radius therefore gives the values shown below.

Quantity	Result	Meaning
$V(T_0)$	5,240,571.546 m^3	Main-CV water inventory at 900 s
$V(T_1)$	12,441,821.987 m^3	Main-CV water inventory at 2,592,000 s
$\Delta V = V(T_1) - V(T_0)$	+7,201,250.441 m^3	30-day increase in Main-CV water storage

Therefore the Water LHS for the selected 30-day interval is ΔV = +7,201,250.441 m^3, or approximately $+7.20 \times 10^6$ m^3. The positive sign indicates an increase in stored water within the Main CV.

Walkthrough interpretation. The added boxes show the actual contents of the key variables after each function, so the calculation can be followed from model IDs and node coordinates to one real element volume and finally to the Main-CV storage change.